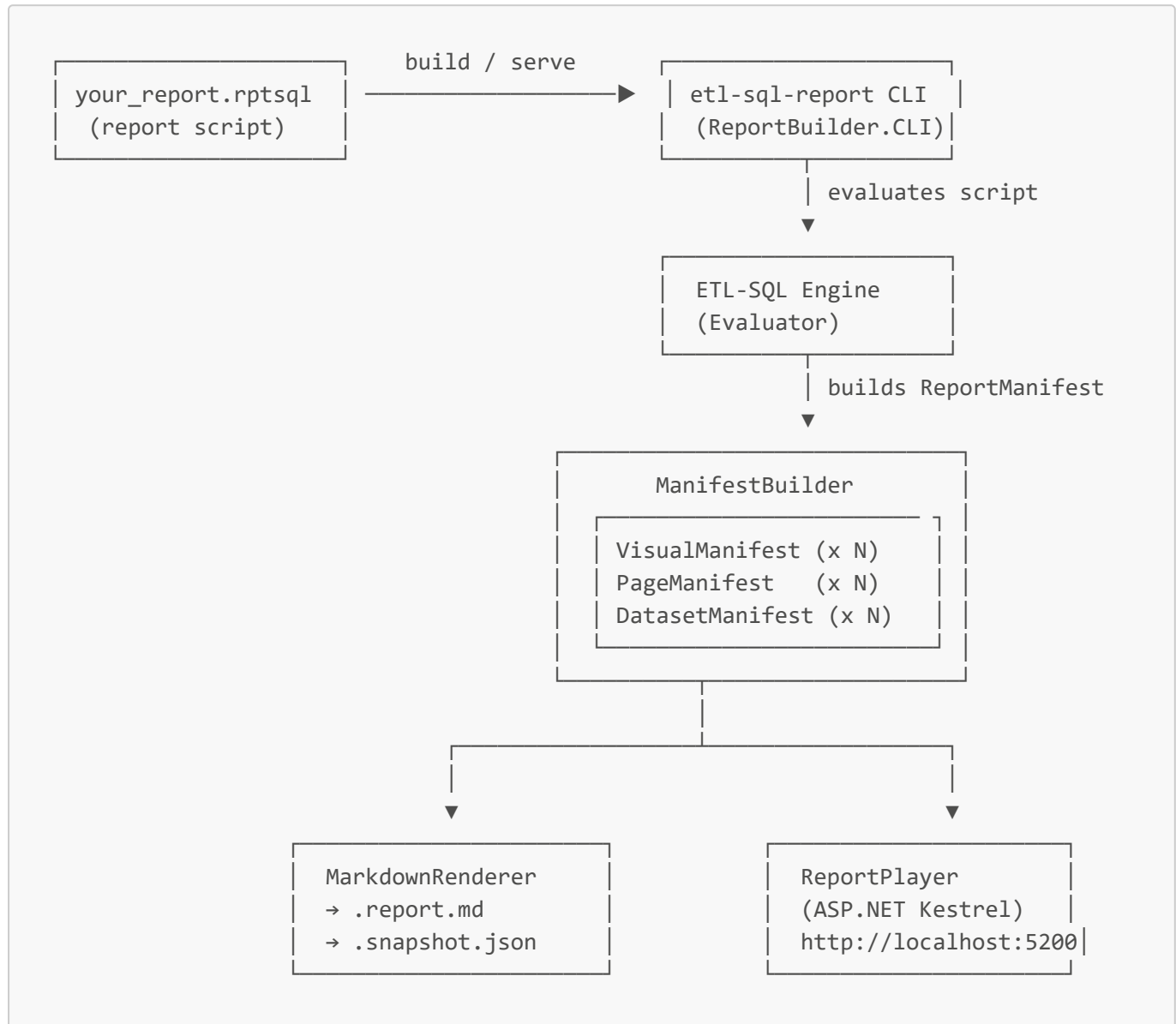


Report-SQL Scripting Guide

Report-SQL extends ETL-SQL with three new statement types — **CREATE DATASET**, **CREATE VISUAL**, and **CREATE PAGE** — plus a CLI build tool and a live web dashboard for building and serving interactive reports.

How it works — architecture overview



A **.rptsql** file is a normal ETL-SQL script that may also contain **CREATE VISUAL**, **CREATE PAGE**, and **CREATE DATASET** statements. The engine evaluates it exactly like any **.etlsql** file; the new statements register visual/page/dataset definitions in the execution context. After evaluation the **ManifestBuilder** snapshots the data and produces a **ReportManifest** — a serialisable JSON structure consumed by both the static Markdown renderer and the live web dashboard.

The @@DATASET System Variable

Report-SQL provides the **@@DATASET** system variable which can be used to manually pass data to visuals or capture the result of a **CREATE DATASET** operation. It is treated as a **LIST** of rows. You can manually assign to

it before a visual is declared if that visual uses it as a source:

```
DECLARE @@DATASET = (('Product A', 100), ('Product B', 250));
CREATE VISUAL ManualCard AS CARD (SOURCE = @@DATASET, MAPPINGS(VALUE = Col1, LABEL
= Col0));
```

Quick start

```
-- 1. Connect and pull data
CREATE CONNECTION c ON FLATFILE('data/sales.csv');

SELECT region, SUM(revenue) AS revenue
INTO #summary
FROM c
GROUP BY region;

-- 2. Define a visual
CREATE VISUAL SalesChart AS BAR (
  SOURCE = #summary,
  MAPPINGS (X = region, Y = revenue),
  OPTIONS (
    X_AXIS (label = 'Region'),
    Y_AXIS (label = 'Revenue ($)')
  )
);

-- 3. Arrange on a page
CREATE PAGE Main AS LAYOUT (
  STRUCTURE = 'grid:1x1',
  MAP ('A' = SalesChart)
);
```

Save as `report.rptsql`, then:

```
etl-sql-report build report.rptsql      # → report.report.md +
report.snapshot.json
etl-sql-report serve report.rptsql      # → opens http://localhost:5200
```

CREATE VISUAL

```
CREATE VISUAL <name> AS <TYPE> (
  SOURCE = <source>,
  [MAPPINGS (role = column, ...),]
```

```
[OPTIONS (key = value, ..., X_AXIS (...), Y_AXIS (...)),]
[ACTIONS (trigger = action, ...)]
);
```

All clauses inside the outer () are separated by commas. The closing) ends the statement. Only SOURCE is required; all other clauses are optional.

Visual types

Type	Description	Chart.js type	Rendered as
BAR	Vertical bar chart. Supports grouping via a SERIES mapping.	bar	Canvas
LINE	Line chart. Supports multiple series and smooth curves.	line	Canvas
SCATTER	X/Y scatter plot. Each row becomes one point.	scatter	Canvas
PIE	Pie / doughnut chart. One slice per row.	pie	Canvas
TABLE	Paginated, scrollable data grid.	—	HTML <table>
CARD	Single large KPI number with an optional label.	—	Styled <div>
SLICER	Parameter dropdown. Interactive in the live dashboard; omitted in static markdown.	—	<select>

SOURCE

The SOURCE clause is **required**. It provides the data for the visual. It can be:

```
-- a) A pre-populated temp table (set earlier in the same script)
SOURCE = #my_temp_table

-- b) An inline SELECT (must be wrapped in parentheses)
SOURCE = (SELECT region, SUM(revenue) AS rev FROM #summary GROUP BY region)
```

Inline SELECTs are evaluated at build time and their results are snapshotted into the manifest. If you need the visual to refresh independently, use CREATE DATASET to define the source table and reference the dataset by name.

MAPPINGS

Maps the columns returned by SOURCE to semantic **roles** expected by the renderer. Roles are case-insensitive.

BAR and LINE

Role	Description
X	Category axis column (string or date). Required.
Y	Value axis column (numeric). Required.
SERIES	Optional grouping column. Each distinct value becomes a separate coloured dataset on the chart.

```
-- Simple: single series
MAPPINGS (X = month, Y = revenue)

-- Grouped: one line/bar per region
MAPPINGS (X = month, Y = revenue, SERIES = region)
```

SCATTER

Role	Description
X	Horizontal axis column (numeric).
Y	Vertical axis column (numeric).

```
MAPPINGS (X = score, Y = rank)
```

PIE

Role	Description
LABEL	Column to use as the slice label.
VALUE	Column to use as the slice size (numeric).

```
MAPPINGS (LABEL = category, VALUE = total)
```

CARD

Role	Description
LABEL	Small caption rendered above the number. If omitted, the column name is used.
VALUE	The primary number or string to display large.

```
MAPPINGS (VALUE = val, LABEL = lbl)
```

SLICER

Role	Description
PARAMETER	The @variable whose value is controlled by this slicer.

```
-- SOURCE should return distinct values for the dropdown
SOURCE = (SELECT DISTINCT region FROM #summary),
MAPPINGS (PARAMETER = @region)
```

TABLE

TABLE visuals use all columns returned by SOURCE in definition order. No MAPPINGS clause is needed.

OPTIONS

General key/value options plus optional axis sub-blocks. All are optional:

```
OPTIONS (
  -- flat options:
  title   = 'Revenue Over Time',  -- overrides the default visual name as the
  chart title
  stacked = true,                  -- BAR / LINE: stack series (true | false)
  smooth  = true,                  -- LINE only: use bezier smoothing (true |
  false)

  -- axis sub-blocks (BAR, LINE, SCATTER only):
  X_AXIS (
    label = 'Month',               -- axis label shown below/beside the axis
    min   = 0,                     -- minimum axis value (numeric)
    max   = 100                    -- maximum axis value (numeric)
  ),
  Y_AXIS (
    label = 'Revenue ($)',
    min   = 0
  )
)
```

Flat OPTIONS reference

Key	Applies to	Values	Description
title	All chart types	Any string	Chart title. Defaults to the visual name.
stacked	BAR, LINE	true / false	Stack series on top of each other.
smooth	LINE	true / false	Smooth curves via bezier interpolation (Chart.js tension).

X_AXIS / Y_AXIS sub-block options

Key	Values	Description
<code>label</code>	Any string	Human-readable axis label.
<code>min</code>	Numeric	Force axis minimum. Useful to prevent the chart from starting above zero.
<code>max</code>	Numeric	Force axis maximum.

ACTIONS

Actions wire up interactive behaviour in the live dashboard. Currently two action types are supported:

```
ACTIONS (
  ON_CLICK = DRILL_DOWN(Target = DetailChart, Key = region),
  ON_CHANGE = SET_PARAMETER(@region, region)
)
```

Trigger	Description
<code>ON_CLICK</code>	Fires when the user clicks a chart element (bar, slice, point).
<code>ON_CHANGE</code>	Fires when a SLICER selection changes.

DRILL_DOWN

```
ON_CLICK = DRILL_DOWN(Target = <VisualName>, Key = <column>)
```

When clicked, passes the selected row's value in `Key` as a filter into the target visual's inline SELECT. The target visual is re-queried with the key value injected into the parameter context.

SET_PARAMETER

```
ON_CHANGE = SET_PARAMETER(@paramName, <columnRef>)
```

Sets the named `@param` to the selected value. Any visual whose inline SELECT references `@paramName` is automatically re-queried. This is the primary mechanism for making slicers drive other visuals.

CREATE DATASET

Pre-computes a named temp table that can be independently refreshed and optionally encrypted or compressed. Use this when multiple visuals share the same expensive base query, or when you want separate refresh cadences.

```
CREATE DATASET #<name>
  [REFRESH EVERY '<interval>']
  [TTL = '<duration>']
  [COMPRESS = ON|OFF]
  [ENCRYPT = ON [KEYFILE = '<path>']]
AS ( SELECT ... );
```

Full example

```
CREATE DATASET #sales_snap
  REFRESH EVERY '1h'
  TTL = '24h'
  COMPRESS = ON
  ENCRYPT = ON KEYFILE = 'C:\keys\report.key'
AS (SELECT region, product, SUM(revenue) AS revenue
     FROM sales
     GROUP BY region, product);
```

CREATE DATASET clause reference

Clause	Required	Description
#<name>	Yes	Temp table name. The # prefix is automatically added if omitted.
REFRESH EVERY '<interval>'	No	Re-compute interval. Format: <n>s, <n>m, <n>h, or <n>d (e.g. '30m', '1h', '7d').
TTL = '<duration>'	No	How long a snapshot stays valid before <code>IsStale</code> returns true. Same interval format.
COMPRESS = ON	No	Compress the snapshot file on disk. Default <code>OFF</code> .
ENCRYPT = ON	No	AES-encrypt the snapshot file. Requires <code>KEYFILE</code> .
KEYFILE = '<path>'	Required when <code>ENCRYPT = ON</code>	Absolute path to the AES key file. Linter raises an error if <code>ENCRYPT = ON</code> but <code>KEYFILE</code> is absent.
AS (SELECT ...)	Yes	The query to execute. Must be the last clause before the semicolon.

CREATE PAGE

Arranges visuals into a named layout. Multiple pages can be defined in one script; the web dashboard renders each as a distinct section.

```
CREATE PAGE <name> AS LAYOUT (
  STRUCTURE = '<layout-string>',
  MAP (
    '<slot>' = VisualName,
    ...
  )
)
[WITH PARAMETERS (@param = default, ...)]
;
```

STRUCTURE string

STRUCTURE is a descriptive hint string passed through to the renderer. The ReportPlayer currently reads it to understand the intended layout intent.

Value	Effect
'grid:1x1'	Single full-width visual.
'grid:1x2'	One column, two rows.
'grid:2x1'	Two columns, one row.
'grid:2x2'	Two columns, two rows.
'grid:2x3'	Two columns, three rows.
'grid:3x2'	Three columns, two rows.

Note (known issue): The grid layout hint is currently passed to the frontend but the runtime renders visuals in a single flowing column. Full CSS grid rendering is tracked in the roadmap.

MAP

MAP assigns each visual to a slot. Slots are single-quoted letter strings ('A', 'B', ...). Visuals are rendered in ascending alphabetical slot order.

```
MAP (
  'A' = KpiCard,
  'B' = BarChart,
  'C' = LineChart,
  'D' = DataTable
)
```

WITH PARAMETERS

Declares page-level parameters with optional default values. These are exposed as slicers in the live dashboard and can be overridden by **SET_PARAMETER** actions.


```
WITH PARAMETERS (@region = 'All', @year = '2024')
```

When a parameter is changed by a slicer or **DRILL_DOWN**, the DashboardService re-evaluates the entire script with **DECLARE @region = '...'**; prepended, so every inline SELECT in every visual re-runs automatically.

Full page example

```
CREATE PAGE Overview AS LAYOUT (  
  STRUCTURE = 'grid:2x3',  
  MAP (  
    'A' = TotalRevenue,  
    'B' = RegionFilter,  
    'C' = RevenueByRegion,  
    'D' = RevenueByProduct,  
    'E' = UnitsByMonth,  
    'F' = SalesTable  
  )  
)  
WITH PARAMETERS (@region = 'All');
```

If no **CREATE PAGE** statements exist, all visuals are rendered in definition order.

CLI — etl-sql-report

The CLI is the entry point for building and serving reports outside of VS Code.

build

Evaluates the script, builds a **ReportManifest**, and writes output files:

```
etl-sql-report build report.rptsql  
etl-sql-report build report.rptsql --output out/dashboard.md  
etl-sql-report build report.rptsql --format json
```

Output files produced:

File	Description
<script>.report.md	GitHub Flavored Markdown document with embedded Chart.js config comments. Default when --format md .
<script>.report.json	Raw manifest JSON. Default when --format json .
<script>.snapshot.json	Snapshot of all visual data rows and metadata, always written alongside the report.

Flags:

Flag	Default	Description
<code>--output, -o</code>	<code><script>.report.md</code> or <code>.report.json</code>	Override the output file path.
<code>--format, -f</code>	<code>md</code>	Output format: <code>md</code> (Markdown) or <code>json</code> (raw manifest).

How the Markdown report works:

- `TABLE` visuals → GFM pipe table (capped at 1,000 rows).
- `CARD` visuals → Blockquote with bold label and value.
- `BAR`, `LINE`, `SCATTER`, `PIE` → HTML comment `<!-- CHART:{...} -->` containing the Chart.js JSON config, followed by a fallback GFM table of the raw data. The VS Code preview panel and `etl-sql-report serve` process this comment to render the interactive chart.
- `SLICER` visuals → Noted as `[Slicer – interactive only]`.

refresh

Re-evaluates the script and updates the snapshot file without writing a new report document. Use this to keep the snapshot fresh on a schedule:

```
etl-sql-report refresh report.rptsql
```

The snapshot is stored alongside the script as `<script>.snapshot.json`. The ReportPlayer considers the snapshot stale if the script file has been modified since the snapshot was built, or if the TTL (default 24 h) has elapsed. When stale, a banner is shown in the dashboard.

serve

Starts the web dashboard at `http://localhost:5200`:

```
etl-sql-report serve report.rptsql
```

Internally this launches `ETL-SQL.ReportPlayer` (the Kestrel ASP.NET server) and opens the browser after 2.5 s. Keep the process running; the dashboard is served for as long as the process is alive.

ReportPlayer — web dashboard

The ReportPlayer is a lightweight ASP.NET Minimal API server that hosts the report as an interactive dashboard.

Endpoints

Method	Path	Description
GET	/	Serves the full dashboard HTML shell with embedded manifest.
GET	/api/manifest	Returns the current <code>ReportManifest</code> as JSON.
GET	/api/refresh	Triggers a full rebuild (re-evaluates the script) and returns <code>{ rebuilt: true, visuals: N }</code> .
POST	/api/parameter	Updates one parameter and triggers a rebuild. Body: <code>{ "name": "@region", "value": "West" }</code> .

Slicer interactivity

When a user changes a **SLICER** dropdown in the dashboard:

1. The browser calls `POST /api/parameter` with the parameter name and selected value.
2. `DashboardService.SetParameterAsync()` prepends `DECLARE @region = 'West';` to the script source and re-evaluates it.
3. The updated manifest is returned and the page re-renders all visuals with the new data.

Staleness banner

If the manifest was built before the script file was last written, or if more than 24 hours have passed, a yellow banner appears:

⚠ Snapshot may be stale — run `etl-sql-report refresh` to update.

You can also hit `/api/refresh` in the browser to force a live rebuild without restarting the server.

Dashboard rendering (report-runtime.js)

The dashboard frontend is a single vanilla-JS file (`wwwroot/report-runtime.js`) that works in two modes:

- **VS Code WebviewPanel:** reads `window.__MANIFEST__` injected by the extension.
- **Web (ReportPlayer):** fetches `/api/manifest` on load.

Visual rendering is handled entirely client-side using [Chart.js v4](#). Chart.js is loaded from CDN; no bundler is required.

VS Code preview

With a `.rptsql` file open, run **ETL-SQL: Preview Report** from the command palette or click the `$(graph)` icon in the editor title bar. A webview panel opens beside the editor and auto-refreshes every time you save the file.

Configuration:

Setting	Default	Description
<code>etlsql.reportCliPath</code>	<i>(empty)</i>	Full path to <code>etl-sql-report.exe</code> . Leave empty to use <code>dotnet run</code> from the source tree in development.

Note: `.rptsql` extension support (syntax highlighting, linting, preview button) is currently tracked as a roadmap item. The preview command works today; dedicated language association for `.rptsql` is pending.

Linters rules (language server)

The language server checks `.rptsql` files automatically:

Rule	Severity	Condition
<code>VisualSourceExists</code>	Warning	<code>SOURCE = #table</code> references a temp table not defined earlier in the script.
<code>VisualMappingColumnExists</code>	Warning	A <code>MAPPINGS</code> role references a column not returned by the <code>SOURCE</code> inline SELECT.
<code>PageVisualReferenced</code>	Warning	A <code>MAP</code> entry references a visual name that is not defined in the script.
<code>DatasetRefreshInterval</code>	Warning	<code>REFRESH EVERY</code> value does not match the <code><n>s/m/h/d</code> format.
<code>DatasetEncryptWithoutKey</code>	Error	<code>ENCRYPT = ON</code> without a <code>KEYFILE</code> clause.
<code>LayerOrder</code>	Warning	A visual references a dataset defined later in the script, or a page references a visual defined later. Forward references are not supported.
<code>InsertColumnCountMismatch</code>	Warning	<code>INSERT INTO</code> omits the column list and the SELECT provides fewer columns than the target table.

ReportManifest JSON schema

The snapshot and API both return this structure:

```
{
  "source": "C:/reports/sales.rptsql", // script path
  "builtAt": "2026-04-12T18:00:00Z", // UTC build timestamp

  "visuals": [
    {
      "name": "RevenueByRegion",
      "visualType": "Bar",
      "chartConfig": "{...}", // Chart.js config JSON string (null
// for TABLE/CARD/SLICER)
      "columns": ["region", "revenue"], // column headers
      "rows": [["East", "12000"], ...], // row data as string arrays
      "options": { // flat options + mapping hints
        "title": "Revenue By Region",
        "mapping:x": "region",
        "mapping:y": "revenue"
      }
    }
  ]
}
```

```

    }
  }
],

"pages": [
  {
    "name":      "Overview",
    "structure": "grid:2x3",
    "slotMap":   { "A": "TotalRevenue", "B": "RevenueByRegion" },
    "parameters": { "@region": "All" }
  }
],

"datasets": [
  {
    "tempTableName":  "#sales_snap",
    "refreshInterval": "1h",
    "ttl":             "24h",
    "lastRefresh":    "2026-04-12T18:00:00Z",
    "rowCount":       4800
  }
]
}

```

Full working example

Source CSV columns: **region**, **product**, **units**, **revenue**, **month**, **date**

```

DROP CONNECTION IF EXISTS c;
CREATE CONNECTION c ON FLATFILE('TestData/test_sales.csv');

-- Shared base dataset (refreshed every hour)
CREATE DATASET #summary
  REFRESH EVERY '1h'
  AS (SELECT month, region, product,
            SUM(units) AS units,
            SUM(revenue) AS revenue
      FROM c
      GROUP BY month, region, product);

-- Region filter slicer
CREATE VISUAL RegionFilter AS SLICER (
  SOURCE = (SELECT DISTINCT region FROM #summary ORDER BY region),
  MAPPINGS (PARAMETER = @region),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@region, region))
);

-- KPI card: total revenue
CREATE VISUAL TotalRevenue AS CARD (
  SOURCE = (SELECT SUM(revenue) AS val, 'Total Revenue' AS lbl

```

```

        FROM #summary
        WHERE @region = 'All' OR region = @region),
    MAPPINGS (VALUE = val, LABEL = lbl)
);

-- Bar chart: revenue by region
CREATE VISUAL RevenueByRegion AS BAR (
    SOURCE = (SELECT region, SUM(revenue) AS revenue
        FROM #summary
        GROUP BY region),
    MAPPINGS (X = region, Y = revenue),
    OPTIONS (
        title = 'Revenue by Region',
        X_AXIS (label = 'Region'),
        Y_AXIS (label = 'Revenue ($)', min = 0)
    )
);

-- Bar chart: multi-series revenue by region and month
CREATE VISUAL RevenueByRegionMonth AS BAR (
    SOURCE = (SELECT month, region, SUM(revenue) AS revenue
        FROM #summary
        GROUP BY month, region),
    MAPPINGS (X = month, Y = revenue, SERIES = region),
    OPTIONS (
        title = 'Revenue by Region (Monthly)',
        stacked = true,
        X_AXIS (label = 'Month'),
        Y_AXIS (label = 'Revenue ($)', min = 0)
    )
);

-- Pie chart: revenue by product
CREATE VISUAL RevenueByProduct AS PIE (
    SOURCE = (SELECT product, SUM(revenue) AS revenue
        FROM #summary
        GROUP BY product),
    MAPPINGS (LABEL = product, VALUE = revenue),
    OPTIONS (title = 'Revenue by Product')
);

-- Line chart: units by month (smooth)
CREATE VISUAL UnitsByMonth AS LINE (
    SOURCE = (SELECT month, SUM(units) AS units
        FROM #summary
        GROUP BY month),
    MAPPINGS (X = month, Y = units),
    OPTIONS (
        title = 'Units Sold by Month',
        smooth = true,
        X_AXIS (label = 'Month'),
        Y_AXIS (label = 'Units Sold', min = 0)
    )
);

```

```

-- Scatter: units vs revenue per product
CREATE VISUAL UnitsVsRevenue AS SCATTER (
  SOURCE = (SELECT SUM(units) AS units, SUM(revenue) AS revenue
            FROM #summary
            GROUP BY product),
  MAPPINGS (X = units, Y = revenue),
  OPTIONS (title = 'Units vs Revenue')
);

-- Detail table: all rows
CREATE VISUAL SalesTable AS TABLE (
  SOURCE = #summary
);

-- Dashboard page
CREATE PAGE Main AS LAYOUT (
  STRUCTURE = 'grid:2x3',
  MAP (
    'A' = TotalRevenue,
    'B' = RegionFilter,
    'C' = RevenueByRegion,
    'D' = RevenueByProduct,
    'E' = UnitsByMonth,
    'F' = SalesTable
  )
)
WITH PARAMETERS (@region = 'All');

```

Roadmap (known gaps)

Item	Status
CSS grid layout rendering in the web dashboard	Not implemented — visuals currently flow single-column
Multi-page navigation (<code>CREATE NAVIGATION</code>)	Roadmap — design draft in TODO.md
<code>.rptsql</code> VS Code language ID registration	Pending — preview button and linting already work
Partial rebuild on parameter change (only affected visuals)	Planned — currently triggers a full script re-evaluation
<code>TABLE</code> pagination controls in the dashboard	Not implemented — all rows rendered
CARD secondary value / delta / trend indicator	Not implemented